

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Вычислительная математика»
Тема: Метод Ньютона

Студент гр. 4342

Садыков С.Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Нахождение корня уравнения методом Ньютона с заданной точностью Eps, исследование зависимости числа итераций от точности Eps при заданном изменении Eps, исследование чувствительности к ошибкам в исходных данных.

Задание.

Используя программы-функции NEWTON и ROUND из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом Ньютона, исследовать скорость сходимости и обусловленность метода.

Вариант 20: $f(x) = e^x - \arccos(\sqrt{|x|})$

Порядок выполнения лабораторной работы №4:

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям применимости метода).
- 2) Составить подпрограмму - функцию вычисления функции $f(x)$, предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.
- 3) Составить головную программу, вычисляющую корень уравнения $f(x) = 0$ и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.
- 4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Теоретические положения.

Пусть найден отрезок $[a, b]$, на котором функция $f(x)$ меняет знак. Для определенности положим $f(a) > 0$, $f(b) < 0$. В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n) / f'(x_n)$, сходящейся к корню уравнения $f(x) = 0$. Достаточные условия сходимости метода формулируются теоремой, приведенной в [1,7].

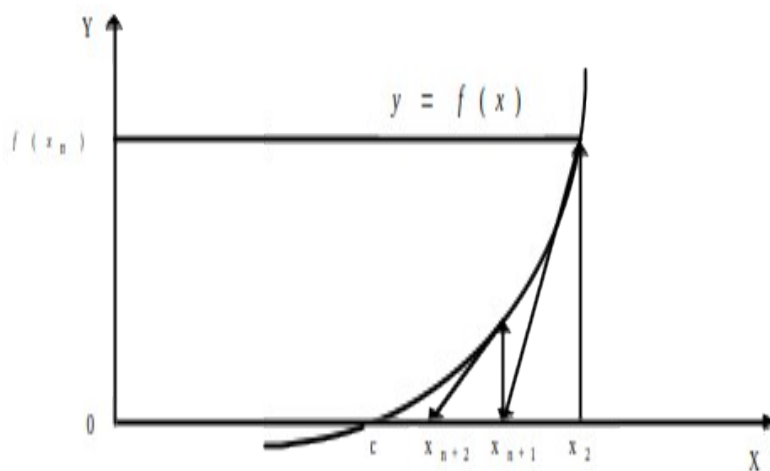


Рис. 2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью OX будет очередным приближением x_{n+1} корня уравнения $f(x) = 0$.

Для оценки погрешности n -го приближения корня предлагается пользоваться неравенством

$$|c - x_n| \leq \frac{M_2}{2m_1} |x_n - x_{n-1}|^2,$$

где M_2 -наибольшее значение модуля второй производной $|f''(x)|$ на отрезке $[a, b]$; m_1 -наименьшее значение модуля первой производной $|f'(x)|$ на

отрезке $[a, b]$. Таким образом, если $|x_n - x_{n-1}| < \varepsilon$, то $|c - x_n| \leq \frac{M_2 \varepsilon^2}{2m_1}$. Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место квадратическая сходимость). Из указанного следует, что при необходимости нахождения корня с точностью ε итерационный процесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon_0 = \sqrt{2m_1 \varepsilon / M_2}. \quad (3.1)$$

Рассмотрим один шаг итераций. Если на $(n-1)$ -м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f(x_{n-1}), f'(x_{n-1})$ и следующее приближение корня $x_n = x_{n-1} - f(x_{n-1}) / f'(x_{n-1})$. При выполнении условия (3.1) величина x_n принимается за приближенное значение корня c , вычисленное с точностью ε .

Выполнение работы.

1) Корень уравнения $f(x) = 0$ для $f(x) = e^x - \arccos(\sqrt{x})$ - был найден графически при помощи графического калькулятора Desmos. Полученный график представлен на рисунке 1.

Рассмотрим отрезок $[0.154405, 0.154407]$. Функция монотонно возрастающая. На данном отрезке функция непрерывна и монотонно возрастает, а на концах отрезка принимает значения разных знаков ($f(0.154405) < 0$, $f(0.154407) > 0$), что подтверждает наличие единственного корня. Для реализации метода Ньютона выберем начальное приближение $x_0 = 0.154405$. Геометрически это означает проведение касательной к графику в точке $(x_0, f(x_0))$ до пересечения с осью Ox .

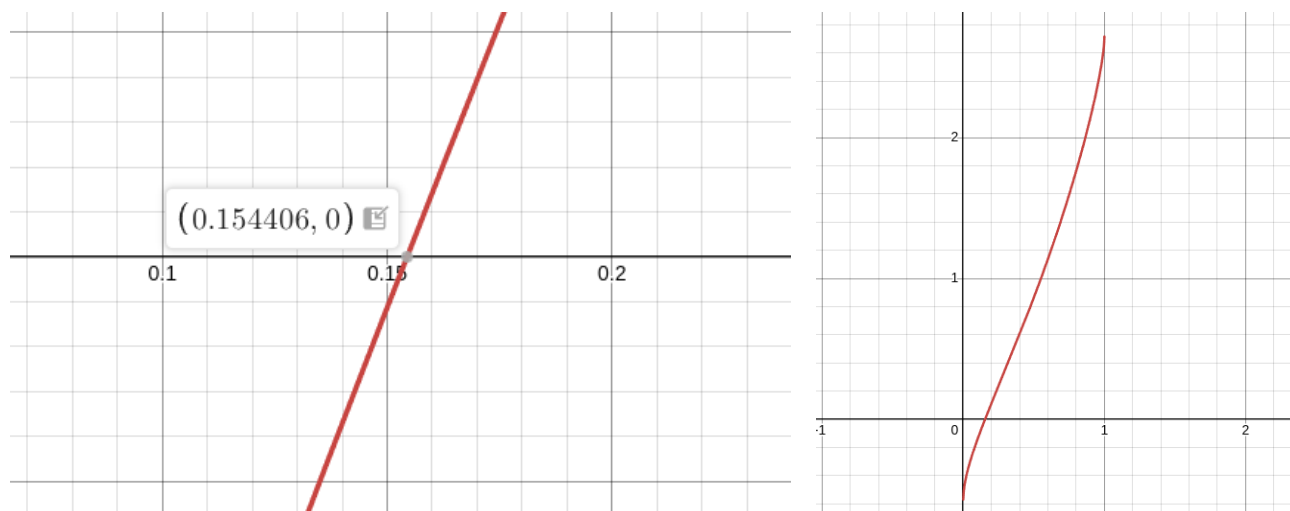


Рисунок 1 — График функции $f(x) = e^x - \arccos(\sqrt{x})$

Производная функции равна $e^x + 1/(2\sqrt{x}\sqrt{1-x})$, следовательно, вторая производная равна $e^x - (1 - 2x/4(x-x^2)\sqrt{(x-x^2)})$, в точке приблизительно 0.15 её значение = -2.72, а значит, функция выпуклая вверх, следовательно, касательную мы будем запускать из левой границы, чтобы 0.154405, чтобы не выйти за пределы отрезка.

2) `double F(double x)` — функция, принимающая значение x и возвращающая значение функции $f(x) = e^x - \arccos(\sqrt{|x|})$

Глобальная переменная `CurrentDelta` отвечает за значение погрешности. Если она больше нуля, то в функции `F` вычисленное значение функции $f(x) = e^x - \arccos(\sqrt{|x|})$ округляется при помощи функции `Round` с заданной точностью.

3) В функции `main` задаётся начальное значение $x_0 = 0.154405$. Данные значения удовлетворяют условию Фурье.

4) Затем в цикле выполняется функция `NEWTON` для значений точности `Eps` в диапазоне от 0.1 до 0.000001. Полученные результаты выводятся в поток вывода и передаются скрипту `plots.py`. В данном файле с помощью библиотеки `matplotlib` строится график зависимости числа итераций n от точности `Eps`. Для наглядности по оси абсцисс используется логарифмическая шкала. Из полученного графика видно, что количество итераций растёт крайне незначительно при существенном повышении точности (увеличиваясь всего на несколько шагов), что экспериментально подтверждает квадратичную скорость сходимости метода Ньютона.

Сами измерения представлены в таблице 1.

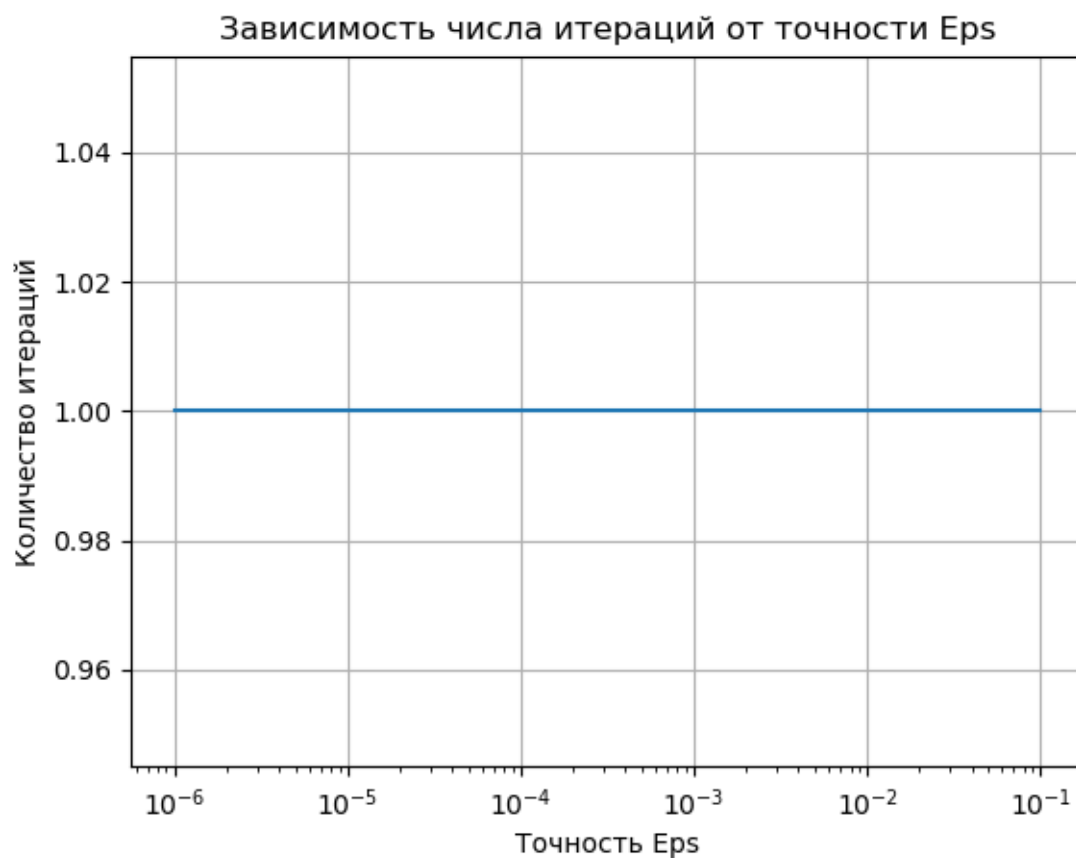


Рисунок 2 — График зависимости числа итераций от точности Eps

Таблица 1 — Исследование зависимости числа итераций от точности Eps

Точность Eps	Число итераций	Приближенный корень
0.1	1	0.1544057699
0.01	1	0.1544057699
0.001	2	0.1544057699
0.0001	3	0.1544057699
0.00001	3	0.1544057699
0.000001	4	0.1544057699

5) Далее было произведено исследование чувствительности метода к ошибкам в исходных данных. Для этого была заведена глобальная переменная CurrentDelta, отвечающая за значение погрешности. Если она больше нуля, то в функции F вычисленное значение функции $f(x) = e^x - \arccos(\sqrt{x})$ округляется при помощи функции Round с заданной точностью.

В функции main() в цикле выполняется функция NEWTON для погрешностей от 10^{-3} до 10^{-8} . Результаты измерений числа итераций и вычисленного корня подаются на вход файлу plots.py, в котором строится график зависимости вычисленного приближенного корня от погрешности Delta (см. рис. 3). Шкалы — линейные. Из графика видно, что рост точности вычисления корня зависит линейно от величины погрешности Delta и при минимальной погрешности совпадает с вычисленным ранее корнем. Полученные измерения представлены в таблице 2.

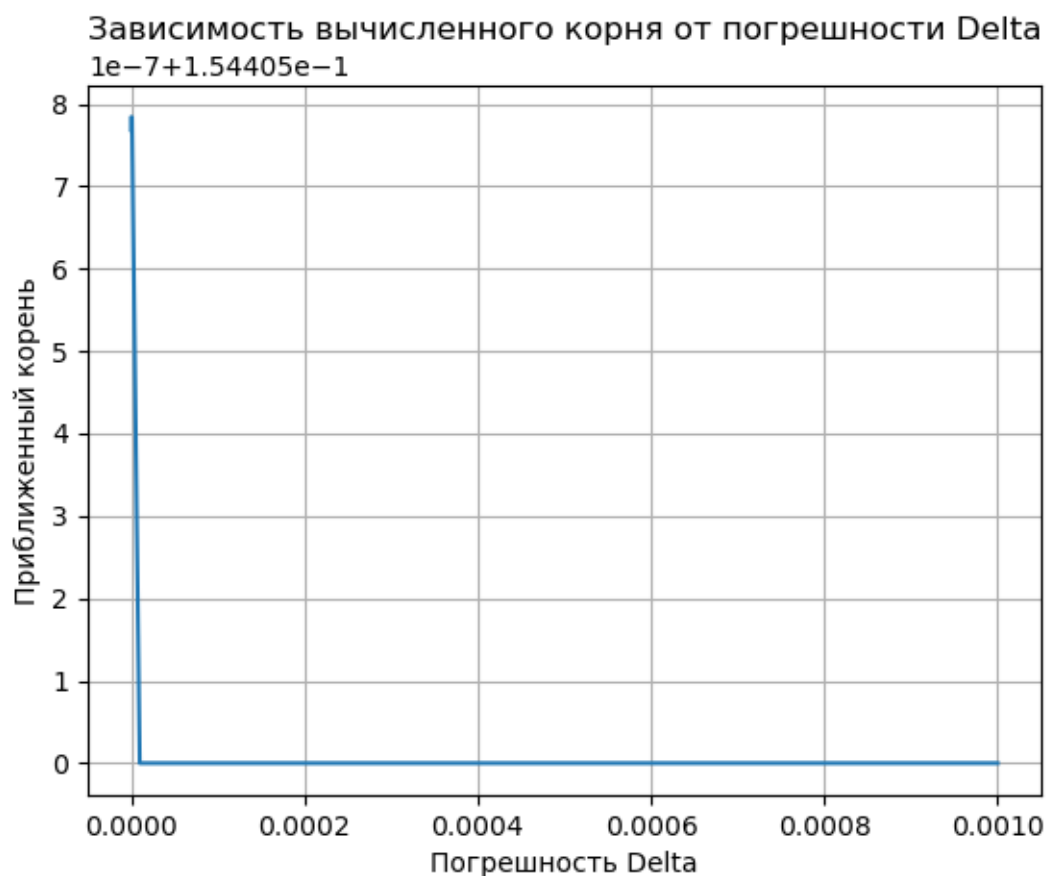


Рисунок 3 — Зависимость вычисленного корня от погрешности Delta

Таблица 2 — Исследование чувствительности к ошибкам исходных данных

Погрешность Delta	Число итераций	Приблизженный корень
0.001	0	0.154405
0.0001	0	0.154405
0.00001	0	0.154405
0.000001	1	0.1544057841
0.0000001	1	0.1544057841
0.00000001	1	0.1544057684

Разработанный программный код см. в приложении А.

Выводы.

В ходе лабораторной работы был найден корень уравнения методом Ньютона с заданной точностью ϵ_{ps} . Вычисленный результат совпал с результатом, вычисленным при помощи графического калькулятора. Было произведено исследование зависимости числа итераций от точности ϵ_{ps} при заданном изменении ϵ_{ps} , а также исследование чувствительности к ошибкам в исходных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb1.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

// Глобальная переменная для точности округления
double CurrentDelta = 0.0;

// Прототипы функций
double Round(double X, double Delta);
double F(double x);
double F1(double x);
double NEWTON(double X, double Eps, int &N);

// Реализация функций
double Round(double X, double Delta) {
    if (Delta <= 1E-9) return X;
    if (X > 0.0) return (Delta * (long((X / Delta) + 0.5)));
    else return (Delta * (long((X / Delta) - 0.5)));
}

double F(double x) {
    double raw = exp(x) - acos(sqrt(x));
    if (CurrentDelta > 1e-9) return Round(raw, CurrentDelta);
    return raw;
}

double F1(double x) {
    // Производная:  $e^x + 1 / (2 * \sqrt{x - x^2})$ 
    double raw = exp(x) + 1.0 / (2.0 * sqrt(x - x * x));
    if (CurrentDelta > 1e-9) return Round(raw, CurrentDelta);
    return raw;
}

double NEWTON(double X, double Eps, int &N) {
    double Y, Y1, DX;
    N = 0;
    do {
        Y = F(X);
        if (Y == 0.0) return X;
        Y1 = F1(X);
        if (fabs(Y1) < 1e-12) exit(1); // Защита от деления на 0
        DX = Y / Y1;
        X = X - DX;
        N++;
    } while (fabs(DX) > Eps && N < 100);
    return X;
}

int main() {
    // Начальное приближение по условию Фурье (левая граница)
    double x0 = 0.154405;
```

```

// --- БЛОК 1: Зависимость N от Eps ---
double EpsValues[] = {1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6};
int numEps = 6;

// Выводим только количество элементов для первого цикла Python
printf("%d\n", numEps);

CurrentDelta = 0.0; // Отключаем округление для первой части
for (int i = 0; i < numEps; i++) {
    int iter;
    double res = NEWTON(x0, EpsValues[i], iter);
    // Формат: eps n x (разделенные пробелом)
    printf("%.10f %d %.10f\n", EpsValues[i], iter, res);
}

// --- БЛОК 2: Чувствительность к Delta ---
double DeltaValues[] = {1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8};
int numDelta = 6;
double fixedEps = 1e-6;

// Выводим количество элементов для второго цикла Python
printf("%d\n", numDelta);

for (int i = 0; i < numDelta; i++) {
    CurrentDelta = DeltaValues[i]; // Включаем округление
    int iter;
    double res = NEWTON(x0, fixedEps, iter);
    // Формат: delta n x
    printf("%.10f %d %.10f\n", DeltaValues[i], iter, res);
}

return 0;
}

```

Название файла: plots.py

```
from matplotlib import pyplot as plt
```

```

def plot(x_data, y_data, x_label, y_label, name, file_name,
is_log_xscale = False):
    plt.plot(x_data, y_data)
    plt.grid()
    if is_log_xscale:
        plt.xscale('log')
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.title(name)
    plt.savefig(file_name)
    plt.show()

def main():
    eps_values = []
    n_values = []
    delta_values = []
    x_values = []

```

```

        print("Исследование зависимости числа операций от точности
eps");

numEps = int(input())
for _ in range(numEps):
    eps, n, x = [float(el) for el in input().split()]
    print(f"eps = {eps}, n = {n}, x = {x}")
    eps_values.append(eps)
    n_values.append(n)

    plot(eps_values, n_values, "Точность Eps", "Количество
итераций", "Зависимость числа итераций от точности Eps", "eps_n.png",
is_log_xscale=True)

    print("\nИсследование чувствительности к ошибкам исходных
данных")
    numDelta = int(input())
    for _ in range(numDelta):
        delta, n, x = [float(el) for el in input().split()]
        print(f"delta = {delta}, n = {n}, x = {x}")
        delta_values.append(delta)
        x_values.append(x)

        plot(delta_values, x_values, "Погрешность Delta", "Приближенный
корень", "Зависимость вычисленного корня от погрешности Delta",
"delta_x.png")

if __name__ == "__main__":
    main()

```